

Discussion 4

Linear Regression and NN Backpropagation

This week in AI



Linear Regression

Overview of Linear Regression

Linear Regression

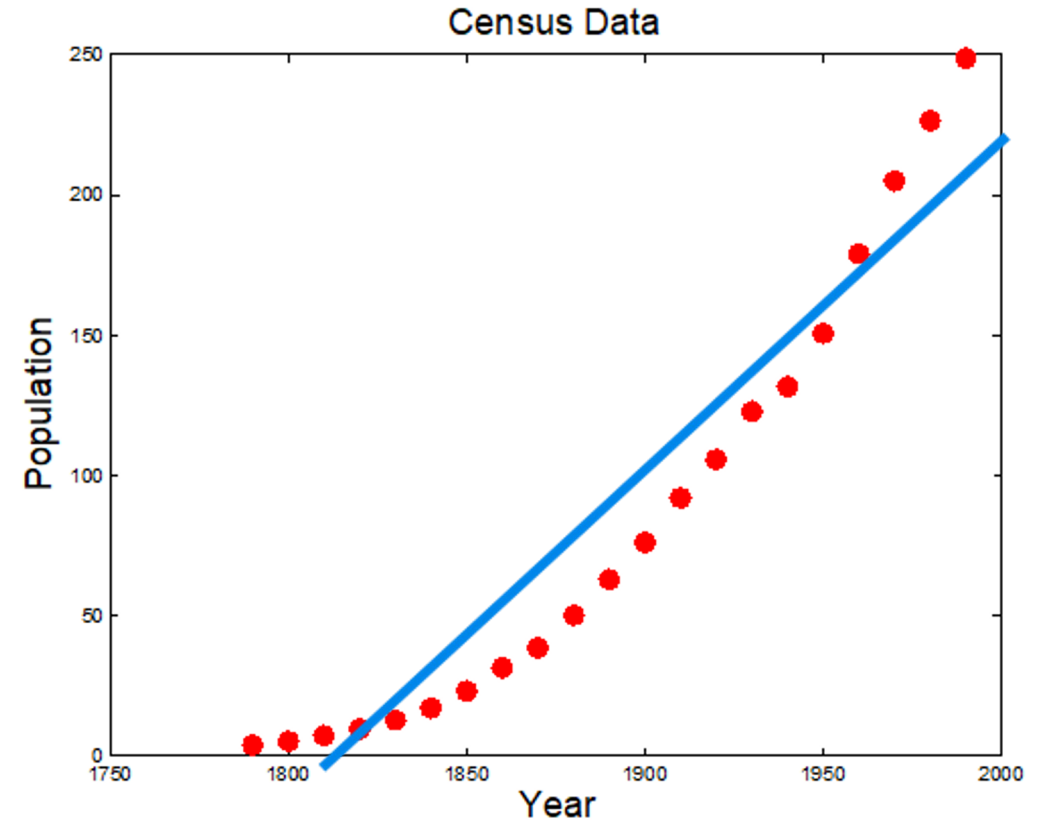
- Model for regression problems
- Inputs are numerical values, output is a numerical value
- We fit a line to our data:

$$y = w_0 + w_1 x + \varepsilon$$

Diagram illustrating the components of the linear regression equation:

- w_0 is labeled as the **intercept** (purple box).
- w_1 is labeled as the **slope** (orange box).
- ε is labeled as the **error** (green box).

- Intuitively, think line of best fit



Linear Regression

- Model for regression problems
- Inputs are numerical values, output is a numerical value
- We fit a line to our data:

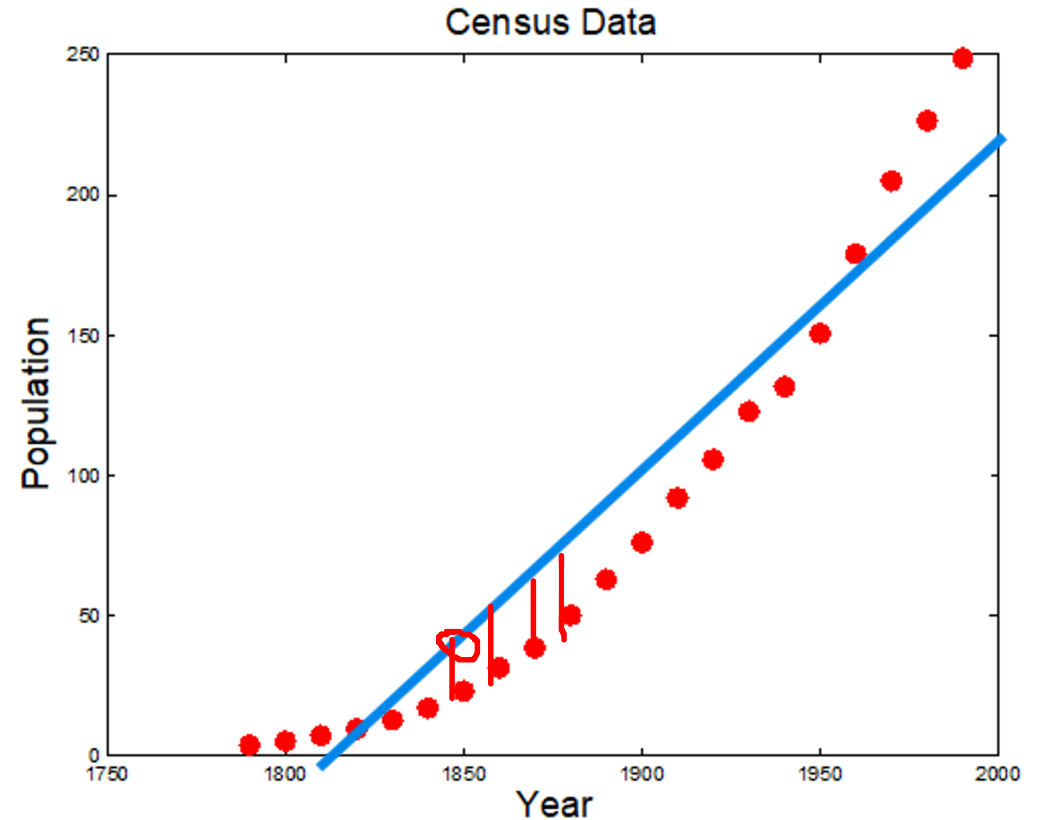
$$y = w_0 + w_1x + \varepsilon$$

Diagram illustrating the components of the linear regression equation:

- w_0 is labeled as the **intercept** (purple box).
- w_1 is labeled as the **slope** (orange box).
- ε is labeled as the **error** (green box).

- Intuitively, think line of best fit

How do we compute the model parameters w_0 , w_1 ?



Linear Regression Weights

Finding Linear Regression Weights

- We model our outputs y using function $h_w(x)$ with model weights $\mathbf{w}$
- We want to find the model weights that ***minimize*** the ***sum of the squares of the errors (SSE)***
- We can solve for the weights in two ways:
 - Analytically, by computing the closed-form solution
 - Gradient descent, by iteratively updating the weights until they converge

Analytic Solution

Given our data points, we can set up matrix X (each row is a data point, with 1 as the intercept) and column vector y to solve for the weights:

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_{1,1} & \dots & x_{1,k} \\ \vdots & \vdots & \dots & \vdots \\ 1 & x_{n,1} & \dots & x_{n,k} \end{bmatrix}$$

$$\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

Analytical Solution

Given our data points, we can set up matrix X (each row is a data point, with 1 as the intercept) and column vector y to solve for the weights:

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Great! So we have a closed form solution to find our model weights! We can use this to solve any linear regression problem now, right?!

Not quite. Why might we *not* want to use the analytical solution?

Analytical Solution

Given our data points, we can set up matrix X (each row is a data point, with 1 as the intercept) and column vector y to solve for the weights:

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Great! So we have a closed form solution to find our model weights! We can use this to solve any linear regression problem now, right?!

Not quite. Why might we **not** want to use the analytical solution?

Finding a matrix inverse is **computationally expensive** and highly dependent on our data (collinear data points make it hard to find the inverse).

Analytical Solution

Given our data points, we can set up matrix X (each row is a data point, with 1 as the intercept) and column vector y to solve for the weights:

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Let's see a faster and more generic method of computing the model weights...

Great! So we have a closed form solution to find our model weights! We can use this to solve any linear regression problem now, right?!

Not quite. Why might we *not* want to use the analytical solution?

Finding a matrix inverse is **computationally expensive** and highly dependent on our data (collinear data points make it hard to find the inverse).

Gradient Descent

- For learning rate α (alpha), we can update our weights based on a single data point (this is called stochastic gradient descent):

$$w_0 \leftarrow w_0 + \alpha \left(y - h_w(x) \right)$$

$$w_1 \leftarrow w_1 + \alpha \left(y - h_w(x) \right) x$$

$$w_0 \leftarrow w_0 + \alpha (y - h_w(x))$$
$$w_1 \leftarrow w_1 + \alpha (y - h_w(x)) x$$

Weight Update Example

We want to predict a person's salary based on their years of work experience, given the data. Suppose our current model is:

$$h_w(x) = 60 + 6x$$

What are the new weights after performing one iteration of stochastic gradient descent based on Observation 1? Use $\alpha=0.1$

Observation	Years Work Experience	Salary (in thousands)
1	2	\$60
2	13	\$150
3	25	\$190

Weight Update Example

We want to predict a person's salary based on their years of work experience, given the data. Suppose our current model is:

$$h_w(x) = 60 + 6x$$

What are the new weights after performing one iteration of stochastic gradient descent based on Observation 1? Use $\alpha=0.1$

$$w_0 \leftarrow w_0 + \alpha(y - h_w(x))$$
$$w_1 \leftarrow w_1 + \alpha(y - h_w(x))x$$

Observation	Years Work Experience	Salary (in thousands)
1	2	\$60
2	13	\$150
3	25	\$190

$$w_0 \leftarrow w_0 + \alpha(y - h_w(x))$$
$$w_1 \leftarrow w_1 + \alpha(y - h_w(x))x$$

Weight Update Example

We want to predict a person's salary based on their years of work experience, given the data. Suppose our current model is:

$$h_w(x) = 60 + 6x$$

What are the new weights after performing one iteration of stochastic gradient descent based on Observation 1? Use $\alpha=0.1$

$$w_0 \leftarrow w_0 + \alpha(y - h_w(x))$$
$$w_1 \leftarrow w_1 + \alpha(y - h_w(x))x$$

Observation	Years Work Experience	Salary (in thousands)
1	2	\$60
2	13	\$150
3	25	\$190

$$w_0 \leftarrow 60 + 0.1(y - h_w(x))$$

$$w_1 \leftarrow 6 + 0.1(y - h_w(x))x$$

Weight Update Example

We want to predict a person's salary based on their years of work experience, given the data. Suppose our current model is:

$$h_w(x) = 60 + 6x$$

What are the new weights after performing one iteration of stochastic gradient descent based on Observation 1? Use $\alpha=0.1$

$$w_0 \leftarrow w_0 + \alpha(y - h_w(x))$$
$$w_1 \leftarrow w_1 + \alpha(y - h_w(x))x$$

Observation	Years Work Experience	Salary (in thousands)
1	2	\$60
2	13	\$150
3	25	\$190

$$w_0 \leftarrow 60 + 0.1(60 - h_w(2))$$
$$w_1 \leftarrow 6 + 0.1(60 - h_w(2))2$$

Weight Update Example

We want to predict a person's salary based on their years of work experience, given the data. Suppose our current model is:

$$h_w(x) = 60 + 6x$$

What are the new weights after performing one iteration of stochastic gradient descent based on Observation 1? Use $\alpha=0.1$

$$w_0 \leftarrow w_0 + \alpha(y - h_w(x))$$
$$w_1 \leftarrow w_1 + \alpha(y - h_w(x))x$$

Observation	Years Work Experience	Salary (in thousands)
1	2	\$60
2	13	\$150
3	25	\$190

$$w_0 \leftarrow 60 + 0.1(60 - h_w(2))$$

$$w_1 \leftarrow 6 + 0.1(60 - h_w(2))2$$

$$h_w(2) = 60 + 6 \cdot 2 = 60 + 12 = 72$$

Weight Update Example

We want to predict a person's salary based on their years of work experience, given the data. Suppose our current model is:

$$h_w(x) = 60 + 6x$$

What are the new weights after performing one iteration of stochastic gradient descent based on Observation 1? Use $\alpha=0.1$

$$w_0 \leftarrow w_0 + \alpha(y - h_w(x))$$
$$w_1 \leftarrow w_1 + \alpha(y - h_w(x))x$$

Observation	Years Work Experience	Salary (in thousands)
1	2	\$60
2	13	\$150
3	25	\$190

$$w_0 \leftarrow 60 + 0.1(60 - 72)$$

$$w_1 \leftarrow 6 + 0.1(60 - 72)2$$

Weight Update Example

We want to predict a person's salary based on their years of work experience, given the data. Suppose our current model is:

$$h_w(x) = 60 + 6x$$

What are the new weights after performing one iteration of stochastic gradient descent based on Observation 1? Use $\alpha=0.1$

$$w_0 \leftarrow w_0 + \alpha(y - h_w(x))$$
$$w_1 \leftarrow w_1 + \alpha(y - h_w(x))x$$

Observation	Years Work Experience	Salary (in thousands)
1	2	\$60
2	13	\$150
3	25	\$190

$$w_0 \leftarrow 60 + 0.1(-12)$$

$$w_1 \leftarrow 6 + 0.1(-12)2$$

Weight Update Example

We want to predict a person's salary based on their years of work experience, given the data. Suppose our current model is:

$$h_w(x) = 60 + 6x$$

What are the new weights after performing one iteration of stochastic gradient descent based on Observation 1? Use $\alpha=0.1$

$$w_0 \leftarrow w_0 + \alpha(y - h_w(x))$$
$$w_1 \leftarrow w_1 + \alpha(y - h_w(x))x$$

Observation	Years Work Experience	Salary (in thousands)
1	2	\$60
2	13	\$150
3	25	\$190

$$w_0 \leftarrow 58.8$$

$$w_1 \leftarrow 3.6$$

Working with Linear Regression Models

Interpreting Model Relevance

- Once we have a regression line, we can compute how well the model fits the data using the ***coefficient of determination (R^2)***
- Let $\mathbf{SS}_{tot}$ be the ***total sum of squares*** and $\mathbf{SS}_{err}$ (or $\mathbf{SS}_{res}$) be the ***sum of squares, error (residual)***

$$SS_{tot} = \sum_i (y_i - \bar{y})^2$$

$$SS_{err} = \sum_i (y_i - \hat{y})^2 = \sum_i (y_i - h_w(x_i))^2$$

- Then the ***coefficient of determination R^2*** is:

$$R^2 = 1 - \frac{SS_{err}}{SS_{tot}}$$

$$SS_{tot} = \sum_i (y_i - \bar{y})^2$$

$$SS_{err} = \sum_i (y_i - \hat{y})^2 = \sum_i (y_i - h_w(x_i))^2$$

Coefficient of Determination Interpretation

$$R^2 = 1 - \frac{SS_{err}}{SS_{tot}}$$

We want to predict a person's salary based on their years of work experience, given the data. Suppose our coefficient of determination is $R^2=0.86$

Observation	Years Work Experience	Salary (in thousands)
1	2	\$60
2	13	\$150
3	25	\$190

What does the coefficient of determination tell us about how the model fits the data?

Coefficient of Determination Interpretation

$$SS_{tot} = \sum_i (y_i - \bar{y})^2$$
$$SS_{err} = \sum_i (y_i - \hat{y})^2 = \sum_i (y_i - h_w(x_i))^2$$

$$R^2 = 1 - \frac{SS_{err}}{SS_{tot}}$$

We want to predict a person's salary based on their years of work experience, given the data. Suppose our coefficient of determination is $R^2=0.86$

What does the coefficient of determination tell us about how the model fits the data?

Observation	Years Work Experience	Salary (in thousands)
1	2	\$60
2	13	\$150
3	25	\$190

Our model explains about 86% of the variability observed in the target variable. This indicates that our model is a reasonably good fit, and years of experience accurately predicts salary.

Linear Regression Prediction Example

Suppose we have the linear regression model on the right and new testing data.

What are the model predictions and residuals (errors) for the testing data?

$$h_w(x) = 60 + 6x$$

Observation	Years Work Experience	Salary (in thousands)
4	10	\$55
5	30	\$250

Linear Regression Prediction Example

Suppose we have the linear regression model on the right and new testing data.

What are the model predictions and residuals (errors) for the testing data?

Observation 4:

$$h_w(10) = 60 + 6 \cdot 10 = 120$$

$$y - h_w(10) = 55 - 120 = -65$$

$$h_w(x) = 60 + 6x$$

Observation	Years Work Experience	Salary (in thousands)
4	10	\$55
5	30	\$250

Linear Regression Prediction Example

Suppose we have the linear regression model on the right and new testing data.

What are the model predictions and residuals (errors) for the testing data?

$$h_w(x) = 60 + 6x$$

Observation	Years Work Experience	Salary (in thousands)
4	10	\$55
5	30	\$250

Observation 4:

$$h_w(10) = 60 + 6 \cdot 10 = 120$$

$$y - h_w(10) = 55 - 120 = -65$$

Observation 5:

$$h_w(30) = 60 + 6 \cdot 30 = 240$$

$$y - h_w(30) = 250 - 240 = 10$$

From Linear Regression to Neural Networks

Summary: Extending Linear Regression

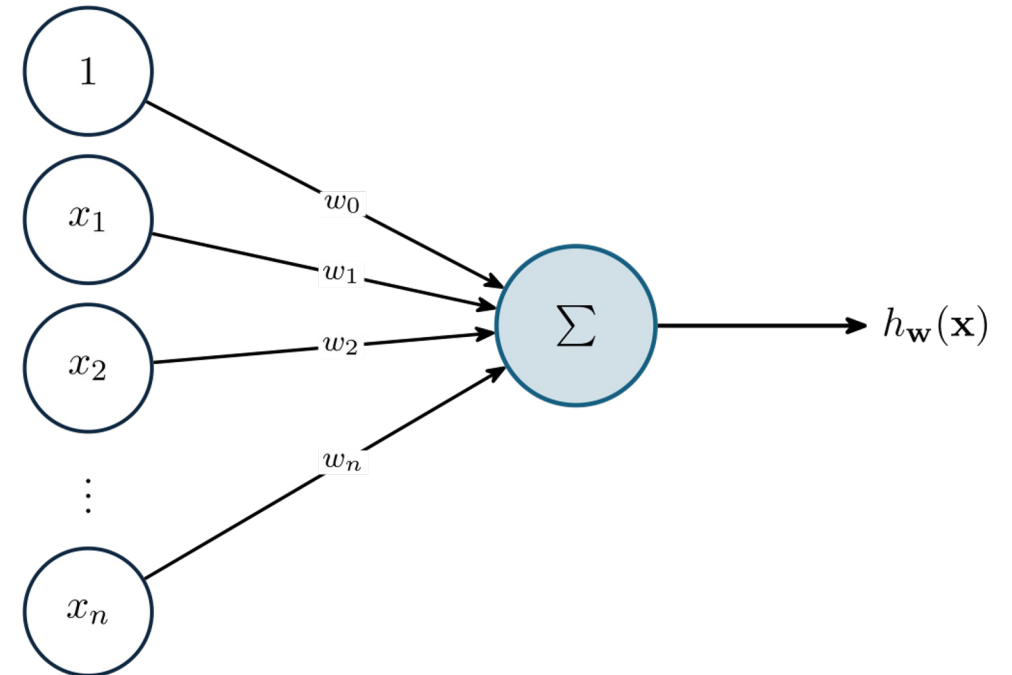
What we covered

- A linear model scores an input by weighting each feature and adding the results
- We fit the weights by minimizing the sum of squared errors
- The closed form is exact, but inverting a matrix is expensive and unstable on collinear data
- Gradient descent nudges the weights one example at a time — cheap and general
- R^2 reports how much of the variance the model explains

$$w_i \leftarrow w_i + \alpha (y - h_{\mathbf{w}}(\mathbf{x})) x_i$$

Where it goes next

That weighted sum is exactly one neuron — linear regression is already a one-unit neural network.



From One Unit to a Network

- A unit takes a weighted sum and then squashes it. That is the only change from linear regression.
- Stacking plain linear layers gains nothing, so every unit applies a nonlinear activation. We use the sigmoid.

one unit

$$z = \mathbf{w} \cdot \mathbf{x} + b = w_1 x_1 + \cdots + w_n x_n + b$$

$$a = g(z)$$

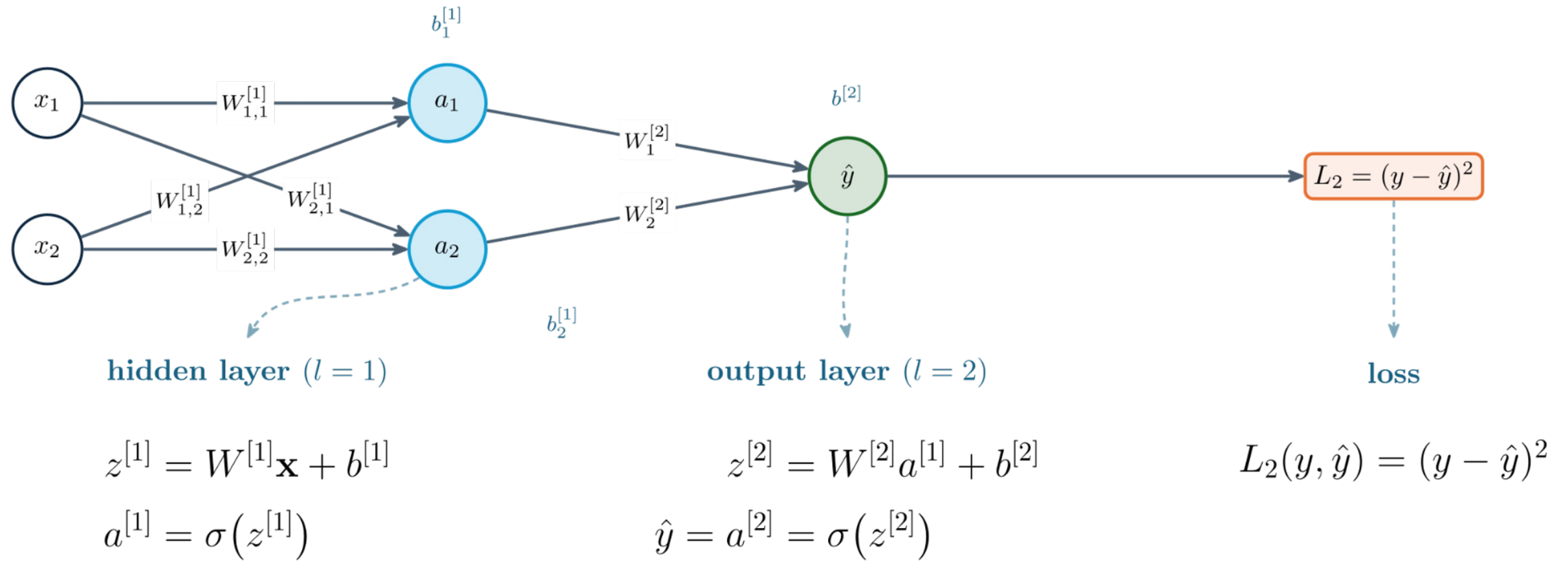
$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

one layer

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}, \quad a^{[l]} = g^{[l]}(z^{[l]}), \quad a^{[0]} = \mathbf{x}$$

$$W^{[2]}(W^{[1]}\mathbf{x}) = (W^{[2]}W^{[1]})\mathbf{x} \implies \text{still one linear map}$$

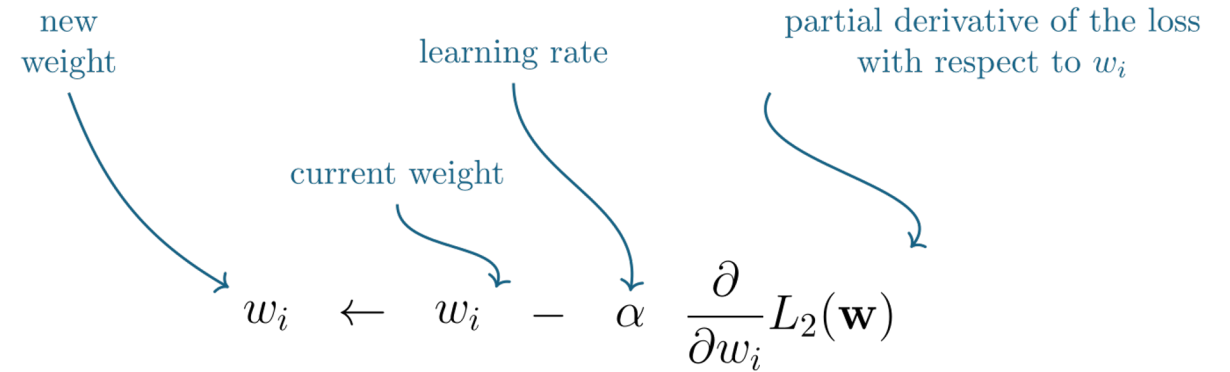
The Network and Its Equations



$W_{j,i}^{[1]}$ is the weight from input x_i into hidden unit a_j • $W_j^{[2]}$ is the weight from a_j into the output unit

Backpropagation: The Weight Update

- Same recipe as linear regression: step every weight downhill.
- The loss is the squared error between the prediction and the target.
- A weight's gradient is a product of partial derivatives — one for each step between that weight and the loss.
- Nothing new to memorise: it is the chain rule.



The diagram shows the weight update equation with three blue arrows pointing to its components:

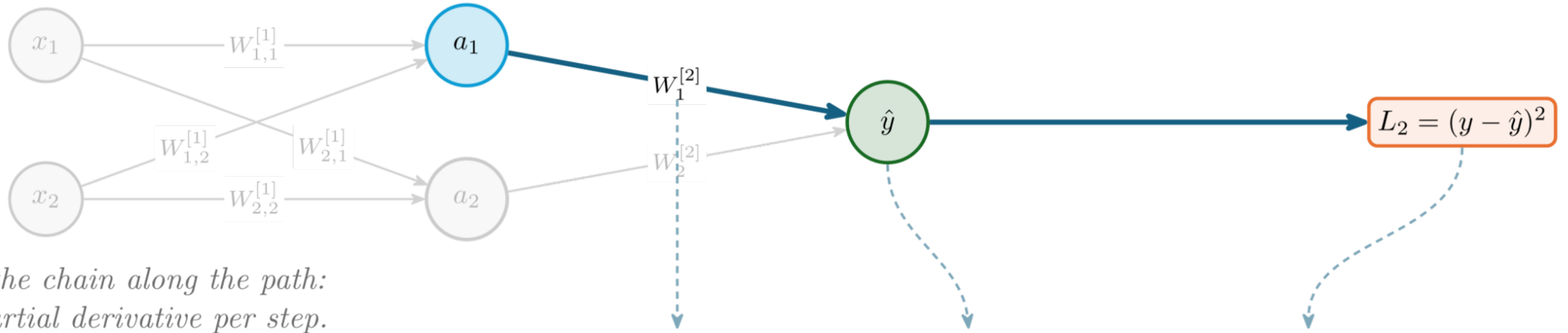
- An arrow from "new weight" points to the leftmost w_i .
- An arrow from "current weight" points to the w_i in the subtraction term.
- An arrow from "learning rate" points to the α .
- An arrow from "partial derivative of the loss with respect to w_i " points to the derivative term $\frac{\partial}{\partial w_i} L_2(\mathbf{w})$.

$$w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} L_2(\mathbf{w})$$

$$L_2(y, \hat{y}) = (y - \hat{y})^2$$

$$\frac{\partial L_2(\mathbf{w})}{\partial w_i} = \underbrace{\frac{\partial L_2(\mathbf{w})}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_i}}_{\text{the chain rule}}$$

Deriving the Update: Output Layer



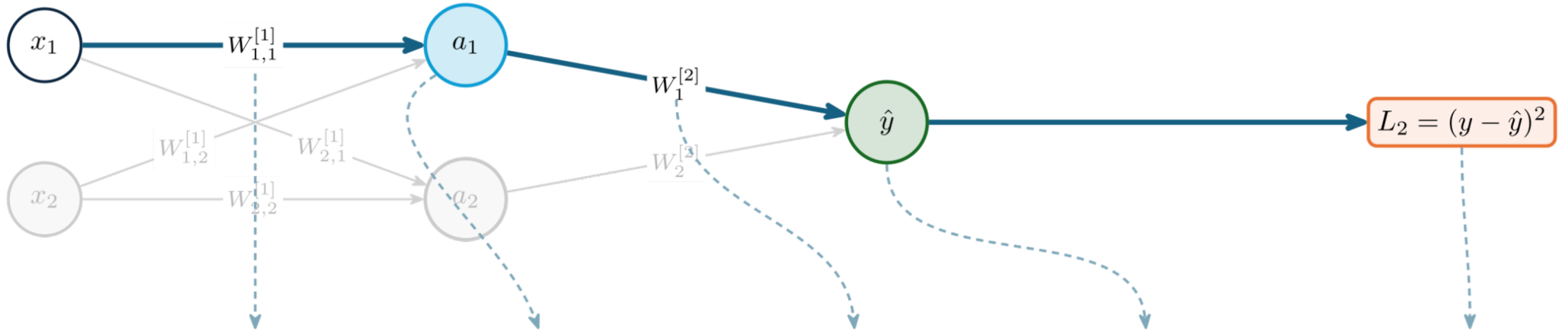
Read the chain along the path:
one partial derivative per step.

$$\frac{\partial L_2}{\partial W_1^{[2]}} = \underbrace{a_1^{[1]}}_{\partial z^{[2]}/\partial W_1^{[2]}} \cdot \underbrace{\hat{y}(1 - \hat{y})}_{\partial \hat{y}/\partial z^{[2]}} \cdot \underbrace{-2(y - \hat{y})}_{\partial L_2/\partial \hat{y}}$$

$$\frac{\partial L_2}{\partial W_1^{[2]}} = -2(y - \hat{y}) \hat{y}(1 - \hat{y}) a_1^{[1]} \quad \text{and} \quad \frac{\partial L_2}{\partial b^{[2]}} = -2(y - \hat{y}) \hat{y}(1 - \hat{y})$$

$$W_1^{[2]} \leftarrow W_1^{[2]} - \alpha \frac{\partial L_2}{\partial W_1^{[2]}}$$

Deriving the Update: Hidden Layer



$$\frac{\partial L_2}{\partial W_{1,1}^{[1]}} = \underbrace{x_1}_{\partial z_1^{[1]} / \partial W_{1,1}^{[1]}} \cdot \underbrace{a_1^{[1]}(1 - a_1^{[1]})}_{\partial a_1^{[1]} / \partial z_1^{[1]}} \cdot \underbrace{W_1^{[2]}}_{\partial z^{[2]} / \partial a_1^{[1]}} \cdot \underbrace{\hat{y}(1 - \hat{y})}_{\partial \hat{y} / \partial z^{[2]}} \cdot \underbrace{-2(y - \hat{y})}_{\partial L_2 / \partial \hat{y}}$$

One layer further back:
two more links, two more factors.

$$\frac{\partial L_2}{\partial W_{1,1}^{[1]}} = -2(y - \hat{y}) \hat{y}(1 - \hat{y}) W_1^{[2]} a_1^{[1]}(1 - a_1^{[1]}) x_1$$

$$W_{1,1}^{[1]} \leftarrow W_{1,1}^{[1]} - \alpha \frac{\partial L_2}{\partial W_{1,1}^{[1]}}$$

Worked Example: The Setup

We predict how well a menu item will be rated from two features, using sigmoid activations on both layers.

Take one stochastic gradient descent step on the **Ice Cream** data point: $x_1 = 2$, $x_2 = 6$, $y = 0.2$, with $\alpha = 0.1$.

What are the new weights and biases?

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad b^{[2]} = \begin{bmatrix} 1 \end{bmatrix}$$

Food	x_1	x_2	Rating
Mousse	1	1	0.1
Pie	3	2	0.5
Ice Cream	2	6	0.2
Sorbet	3	4	0.4
Cheesecake	5	3	0.9

Step 1: Forward Pass

Push Ice Cream through the network, one layer at a time.

$$z^{[1]} = W^{[1]}\mathbf{x} + b^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 6 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$a^{[1]} = \sigma(z^{[1]}) = \begin{bmatrix} 0.73106 \\ 0.73106 \end{bmatrix}$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 0.73106 \\ 0.73106 \end{bmatrix} + \begin{bmatrix} 1 \end{bmatrix} = 2.46212$$

$$\hat{y} = a^{[2]} = \sigma(2.46212) = \mathbf{0.92144}$$

Both hidden units give the same number — they start from identical weights.

Step 2: The Loss and Two Shared Factors

Every weight's gradient begins with the same two partial derivatives.

$$L_2 = (y - \hat{y})^2 = (0.2 - 0.92144)^2 = \mathbf{0.52048}$$

$$\frac{\partial L_2}{\partial \hat{y}} = -2(y - \hat{y}) = -2(0.2 - 0.92144) = 1.44289$$

$$\frac{\partial \hat{y}}{\partial z^{[2]}} = \hat{y}(1 - \hat{y}) = (0.92144)(0.07856) = 0.07239$$

$$\frac{\partial L_2}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{[2]}} = (1.44289)(0.07239) = \mathbf{0.10444}$$

Compute these once; every weight in the network reuses them.

Step 3: The Output Layer

One more factor finishes the chain for the layer-2 weights and bias.

$$\frac{\partial L_2}{\partial W_j^{[2]}} = \underbrace{0.10444}_{\text{from Step 2}} \cdot a_j^{[1]} = (0.10444)(0.73106) = 0.07635$$

$$\frac{\partial L_2}{\partial b^{[2]}} = 0.10444 \cdot 1 = 0.10444$$

$$W_j^{[2]} \leftarrow W_j^{[2]} - \alpha \frac{\partial L_2}{\partial W_j^{[2]}} = 1 - (0.1)(0.07635) = \mathbf{0.99236}$$

$$b^{[2]} \leftarrow b^{[2]} - \alpha \frac{\partial L_2}{\partial b^{[2]}} = 1 - (0.1)(0.10444) = \mathbf{0.98956}$$

The bias behaves like a weight whose input is always 1.

Step 4: Hidden-Layer Gradients

Two more factors reach back to the layer-1 weights and biases.

$$\frac{\partial a_j^{[1]}}{\partial z_j^{[1]}} = a_j^{[1]}(1 - a_j^{[1]}) = (0.73106)(0.26894) = 0.19661$$

$$\frac{\partial L_2}{\partial W_{j,i}^{[1]}} = \underbrace{0.10444}_{\text{from Step 2}} \cdot W_j^{[2]} \cdot \frac{\partial a_j^{[1]}}{\partial z_j^{[1]}} \cdot x_i = (0.10444)(1)(0.19661) x_i = \mathbf{0.02054} x_i$$

$$i = 1 : (0.02054)(2) = 0.04107 \quad i = 2 : (0.02054)(6) = 0.12321$$

$$\frac{\partial L_2}{\partial b_j^{[1]}} = (0.02054)(1) = \mathbf{0.02054}$$

Both hidden units get identical gradients — which is why real networks initialise the weights randomly.

Step 5: Hidden-Layer Update

Same update rule, now with the layer-1 gradients.

$$W_{j,1}^{[1]} \leftarrow W_{j,1}^{[1]} - \alpha \frac{\partial L_2}{\partial W_{j,1}^{[1]}} = 0 - (0.1)(0.04107) = -\mathbf{0.00411}$$

$$W_{j,2}^{[1]} \leftarrow W_{j,2}^{[1]} - \alpha \frac{\partial L_2}{\partial W_{j,2}^{[1]}} = 0 - (0.1)(0.12321) = -\mathbf{0.01232}$$

$$b_j^{[1]} \leftarrow b_j^{[1]} - \alpha \frac{\partial L_2}{\partial b_j^{[1]}} = 1 - (0.1)(0.02054) = \mathbf{0.99795}$$

Every parameter in the network has now moved once.

Step 6: Did the Loss Go Down?

Run the same data point forward again with the updated parameters.

$$z^{[1]} = \begin{bmatrix} -0.00411 & -0.01232 \\ -0.00411 & -0.01232 \end{bmatrix} \begin{bmatrix} 2 \\ 6 \end{bmatrix} + \begin{bmatrix} 0.99795 \\ 0.99795 \end{bmatrix} = \begin{bmatrix} 0.91581 \\ 0.91581 \end{bmatrix} \Rightarrow a^{[1]} = \begin{bmatrix} 0.71419 \\ 0.71419 \end{bmatrix}$$

$$z^{[2]} = (0.99236)(0.71419) + (0.99236)(0.71419) + 0.98956 = 2.40702 \Rightarrow \hat{y} = \sigma(2.40702) = \mathbf{0.91736}$$

$$L_2 = (0.2 - 0.91736)^2 = \mathbf{0.51461} \quad (\text{was } 0.52048)$$

One data point, one step downhill. **Backpropagation is just the chain rule, applied to every weight in the network.**